

Lightweight LLM Fine-Tuning for Query Expansion on MS MARCO

Amir Ghadami

Abstract

This report implements and evaluates Query2Doc query expansion for BM25 retrieval using Qwen2.5-1.5B-Instruct, comparing zero-shot generation, supervised fine-tuning (SFT) on MS MARCO passages, and SFT followed by Direct Preference Optimization (DPO) using retrieval-based preferences. All configurations are evaluated on TREC Deep Learning 2019 through a shared retrieval and scoring pipeline. Zero-shot expansion significantly improves BM25, increasing MAP by 25.0% ($p = 0.0006$) and Recall@1000 by 14.3% ($p = 0.0007$), while its 11.6% nDCG@10 improvement is not statistically significant ($p = 0.065$). Neither SFT nor DPO produces a statistically distinguishable improvement over zero-shot expansion ($p \geq 0.26$), despite both learning their respective training objectives. SFT substantially changes the generated text distribution, while DPO reaches 0.61 training preference accuracy but only 0.52 on held-out pairs. These results provide evidence of objective mismatch: passage likelihood and single-passage retrieval preferences are imperfect proxies for graded retrieval effectiveness. Overall, the results suggest that lightweight Query2Doc expansion can substantially improve lexical retrieval, while additional fine-tuning does not necessarily translate into better retrieval performance.

1 Introduction

BM25 [6] ranks passages based on query-term matching, term rarity, and passage length. Its main limitation is vocabulary mismatch: a query and a relevant passage may express the same information using different terms. For example, a query about *root canal cost* may be answered by a passage discussing *endodontic treatment* without sharing the same vocabulary.

Query2Doc [8] addresses this problem by using a language model to generate a *pseudo-document*: the passage that would answer the query. Its factual accuracy is not the point; what matters is that it contains the vocabulary a relevant passage would use. The generated text is concatenated with the original query before BM25 retrieval. Following the original method, the reformulated query is

$$q' = (q \times 5) + d_{\text{pseudo}}, \quad (1)$$

where repeating the query preserves its contribution relative to the longer pseudo-document.

This report evaluates three configurations using Qwen2.5-1.5B-Instruct [?]. First, zero-shot generation tests whether a lightweight 1.5B model can produce useful expansions. Second, supervised fine-tuning (SFT) on MS MARCO passages tests whether adapting the model to the target corpus improves retrieval. Third, Direct Preference Optimization (DPO) [5] uses BM25 retrieval outcomes to construct preferences between generated expansions. Retrieval effectiveness is not differentiable, so preferences provide a way to optimise toward it indirectly.

The evaluation compares all three configurations with an unexpanded BM25 baseline on TREC Deep Learning 2019. The main findings are that zero-shot expansion significantly improves MAP and Recall@1000, while neither SFT nor DPO produces a statistically distinguishable improvement over zero-shot expansion. The results suggest that adapting the model to the target corpus or optimizing single-passage retrieval preferences does not necessarily translate into improved graded retrieval effectiveness.

2 Method

2.1 Model selection

This report uses Qwen2.5-1.5B-Instruct [7] on an Apple M4 MacBook Pro via the Metal Performance Shaders (MPS) backend. The size was chosen from measured throughput rather than assumed: training in `float32` exceeded available unified memory and ran at 0.094 samples per second, while `bfloat16` reached 2.98 samples per second. At that rate the 1.5B model fits the compute budget with room for the full three-stage pipeline, so the smaller 0.5B alternative was not required. Training uses `bfloat16` rather than `float16` because it retains fp32's exponent range and avoids fp16's overflow instability on MPS. Because `bitsandbytes` is CUDA-only, quantized training is unavailable and both fine-tuning stages use plain LoRA [2].

2.2 Prompt and expansion

The pseudo-document prompt is used verbatim as specified:

```
Write a passage that answers the given query:
Query: {query_text}
Passage:
```

The same string is used for zero-shot generation, as the SFT training template, and for DPO candidate sampling, so that the model is never fine-tuned for a format it does not encounter at test time. Qwen’s chat template is deliberately not applied: the raw completion prompt keeps all three stages consistent and the implementation model-agnostic, at the cost of running an instruction-tuned model outside its intended interface. Generation is capped at 128 new tokens as specified, and expanded queries follow Equation 1.

2.3 Stage 1: zero-shot

The base model generates one pseudo-document per evaluation query with no training. Decoding is greedy, which satisfies the requirement that evaluation-time generation be deterministic or seed-fixed, and does so by construction rather than by seeding. Sampling is used only in Stage 3, where diverse candidates are required.

2.4 Stage 2: supervised fine-tuning

Training examples are (query, relevance-judged passage) pairs assembled from the MS MARCO training queries, qrels and passage collection [4]. What SFT teaches is corpus *style* — passage length, register, and phrasing — rather than factual content; a general instruction-tuned model tends to open with conversational framing that contributes noise terms under BM25.

Loss is computed over the target passage only, with prompt positions masked out. Training on the prompt tokens would spend model capacity reproducing a fixed string that is supplied at inference in any case, and this masking is what distinguishes supervised fine-tuning from continued pretraining on concatenated text. Targets are truncated to 128 tokens, matching the generation cap, so that the model is not trained to produce text longer than it is permitted to generate.

Adaptation uses LoRA [2] on all attention and MLP projections, training 18.5M parameters or 1.18% of the model. Hyperparameters are given in Table 2. The adapter is merged into the base weights after training, so downstream generation and Stage 3 require no adapter-specific handling.

2.5 Stage 3: DPO with retrieval feedback

SFT maximizes the likelihood of one gold passage, which is not the quantity of interest. Retrieval outcome is, and it is not differentiable. Stage 3 approximates it with preferences derived from actual retrieval.

Preference criterion. For each DPO training query, $k = 4$ candidate pseudo-documents are sampled (temperature 0.9, top- p 0.95, seeded), expanded following Equation 1, and used for BM25 retrieval. Each candidate is scored by the reciprocal rank of the known relevant passage:

$$r(d) = \begin{cases} 1/\text{rank}(p^*) & \text{if } p^* \in \text{top-100} \\ 0 & \text{otherwise} \end{cases} \quad (2)$$

where p^* is the passage judged relevant for that query. The highest-scoring candidate becomes the *chosen* response and the lowest-scoring the *rejected* one. Scoring uses depth 100 rather than the depth 1000 used at evaluation, since only the rank of a single known passage is required.

Handling ties. A query yields a preference pair only when the best and worst candidates receive different scores. Two cases produce no usable pair: all candidates rank the relevant passage identically, and no candidate retrieves it at all. Neither expresses a preference, and both are discarded; the discard rate is reported in Section 4.

Training. DPO [5] defines reward implicitly as the log-ratio between the policy and a frozen reference,

$$r_\theta(x, y) = \beta \log \frac{\pi_\theta(y | x)}{\pi_{\text{ref}}(y | x)}, \quad (3)$$

and minimizes $-\log \sigma(r_\theta(x, y_{\text{chosen}}) - r_\theta(x, y_{\text{rejected}}))$. The reciprocal ranks therefore only order each pair; their magnitudes do not enter the loss, a point returned to in Section 6. The reference model is the SFT checkpoint, obtained by disabling the LoRA adapter rather than loading a second copy of the weights. The β term penalizes divergence from that reference, which makes DPO structurally conservative: it refines SFT behaviour rather than reversing it.

2.6 Shared pipeline

Generation, retrieval and evaluation are identical across all configurations; only the checkpoint changes. Retrieval invokes `pyserini.search.lucene` [3] with the parameters fixed by the assignment ($k_1 = 0.9$, $b = 0.4$, depth 1000) against the prebuilt `msmarco-v1-passage` index, and every run is scored by one function that appends to a machine-readable results file. Because the four configurations differ only in the text of their topics file, any measured difference is attributable to the expansion method rather than to how the runs were retrieved or scored.

3 Experimental setup

3.1 Data

Both training and evaluation use the same MS MARCO Passage (V1) collection [4] of 8,841,823 passages. TREC DL 2019 [1] contributes only queries and judgments over that collection; it has no corpus of its own. The two judgment sets differ substantially, and the difference matters for interpreting the results.

	MS MARCO train	TREC DL 2019
Corpus	8.84M passages	the same 8.84M passages
Queries	532,761 judged	43 judged
Judgments / query	≈ 1	≈ 200
Relevance scale	binary (label 1 only)	graded, 0–3
Purpose	training	evaluation

Table 1: Two judgment regimes over one corpus. MS MARCO labels are a by-product of an answer-writing task: the label marks the passage an annotator used, not the output of relevance assessment. TREC DL judgments come from pooling the top results of every participating 2019 system and grading each pooled passage.

The consequence of the first regime is that supervision is shallow. The mean number of positives per training query is approximately 1.0, so multi-positive training is unavailable for most queries. Where several positives exist we retain the first; this is recorded as a simplification in Section 6 rather than presented as a design choice.

DL 2019 released 200 passage topics, of which 43 carry deep judgments. All reported figures are therefore means over 43 queries, and the statistical consequences are treated explicitly in Section 4.

3.2 Splits and leakage

Training queries are partitioned into three disjoint sets by shuffling the full set of judged queries under a fixed seed and slicing. Two properties are asserted in code rather than documented in prose, because a violation of either would silently invalidate every downstream number:

1. No TREC DL 2019 query appears in the training data. Measured overlap: 0 of 43.
2. The SFT, DPO and development splits are mutually disjoint. The assignment requires DPO queries to be drawn from a set disjoint from both the SFT and evaluation queries.

Sorting precedes shuffling, because Python set iteration order is not stable across processes; sampling from an unsorted set would not be reproducible even under a fixed seed.

3.3 Sample-size justification

Both sample sizes were set from measurements taken on the target hardware rather than chosen in advance.

SFT: 8,000 examples. Measured training throughput is 2.98 samples per second, so 8,000 examples is approximately 45 minutes and 500 optimiser steps at an effective batch size of 16. The development loss curve (Figure 3) justifies the size: training loss plateaus near step 100, while held-out loss continues to improve marginally, falling from 2.156 to 2.018 across the full run. The improvement over the final 300 steps is below 0.015. Larger sample sizes were affordable — 30,000 examples would have taken 2.8 hours — but the curve indicates negligible return, and the task is stylistic adaptation rather than knowledge acquisition, which saturates quickly.

DPO: 5,000 queries. Preference construction, not training, dominates the cost of Stage 3. Each query requires $k = 4$ sampled generations of up to 128 tokens plus four BM25 retrievals. Measured cost is 2.30 seconds per query, so 5,000 queries is approximately 3.2 hours. At a measured yield of 72.6% this produces 3,628 preference pairs and 205 optimiser steps.

A smaller pilot of 1,000 queries was run first specifically to obtain the per-query cost and the yield rate before committing to the full run.

A note on retrieval cost. Evaluation retrieval invokes Pyserini as a subprocess, which prints the exact command executed and so allows a reader to verify it against the assignment’s reference commands. This costs roughly ten seconds of JVM startup per call, which is immaterial for four evaluation runs. It is not viable for Stage 3, which performs approximately 20,000 retrievals; that stage uses a persistent `LuceneSearcher` within a single process instead.

3.4 Hyperparameters

	SFT	DPO
Base model	Qwen2.5-1.5B-Instruct	SFT checkpoint
Training examples	8,000 pairs	3,628 preference pairs
Optimiser steps	500	205
LoRA rank / α	16 / 32	16 / 32
LoRA dropout	0.05	0.05
Trainable parameters	18.5M (1.18%)	18.5M (1.18%)
Micro-batch	4	2
Gradient accumulation	4	8
Effective batch	16	16
Learning rate	2×10^{-4}	2×10^{-5}
Warmup	3% of steps	—
Epochs	1	1
Precision	bfloat16	bfloat16
β	—	0.05

Table 2: Training configuration. The effective batch size is held at 16 across both stages so that differences between them are not attributable to batch size. DPO uses a smaller micro-batch because it scores both the chosen and rejected responses under both the policy and the reference, approximately four times the activations of SFT per example.

The DPO learning rate and β in Table 2 are the values from the second of two runs. Both runs are reported in Section 4; the first used 5×10^{-6} and $\beta = 0.1$, the published defaults, and produced no learning.

3.5 Evaluation protocol

All runs are retrieved to depth 1000 and scored with `trec_eval` using the assignment’s reference commands. Three flags carry methodological weight.

- c Scores over every query in the qrels. Without it, a query missing from a run is dropped from the mean rather than scored zero — so a generation failure would *raise* the reported score. Expansion pipelines can drop queries silently, which makes this flag load-bearing.
- q Emits per-query scores in addition to the aggregate. These are stored from the first evaluation onward so that significance testing requires no re-running of experiments.
- l 2 Binarises DL19’s graded judgments at relevance ≥ 2 . Applied to MAP and Recall@1000, which are set-based metrics and require a binary relevant/non-relevant partition. *Not* applied to nDCG@10, which is designed for graded relevance and uses all four levels directly. Grade 1 denotes a passage that is on topic but does not answer the question, which for this task is effectively non-relevant; the standard TREC DL convention therefore excludes it from the set-based metrics.

3.6 Reproducibility

Seeds are fixed for data splitting, model initialisation, training and sampled generation. Evaluation-time generation is greedy and therefore deterministic independent of seeding. The prebuilt index is pinned by artifact identifier (`lucene-inverted.msmarco-v1-passage.20221004.252b5e`) so that a reader can confirm they searched the identical structure. Package versions are pinned in `requirements.txt`.

One reproducibility caveat is worth stating in advance of the results. Two SFT runs with identical seeds, data and hyperparameters produced MAP of 0.3698 and 0.3865, differing only in library version and in the ordering of examples within batches. The 0.017 gap exceeds several of the between-configuration differences reported in Section 4, and we treat it as the effective noise floor of this experiment rather than as a finding.

4 Results

4.1 Retrieval effectiveness

Table 3 reports all four configurations on TREC DL 2019. The unexpanded BM25 row reproduces Pyserini’s published figures exactly (MAP 0.3013, nDCG@10 0.5058, Recall@1000 0.7501), which verifies the index, topics, qrels and scoring path before any model is involved.

	MAP	nDCG@10	R@1000
BM25 (no expansion)	0.3013	0.5058	0.7501
+ zero-shot	0.3765	0.5647	0.8573
+ SFT	0.3865	0.5832	0.8366
+ SFT/DPO	0.3750	0.5780	0.8330
<i>Relative change vs. BM25</i>			
+ zero-shot	+25.0%	+11.6%	+14.3%
+ SFT	+28.3%	+15.3%	+11.5%
+ SFT/DPO	+24.5%	+14.3%	+11.1%

Table 3: Retrieval effectiveness on TREC DL 2019, 43 judged queries, depth 1000. Bold marks the best value per metric. All four rows use the identical index, BM25 parameters and scoring function; the only difference between them is the text of the topics file.

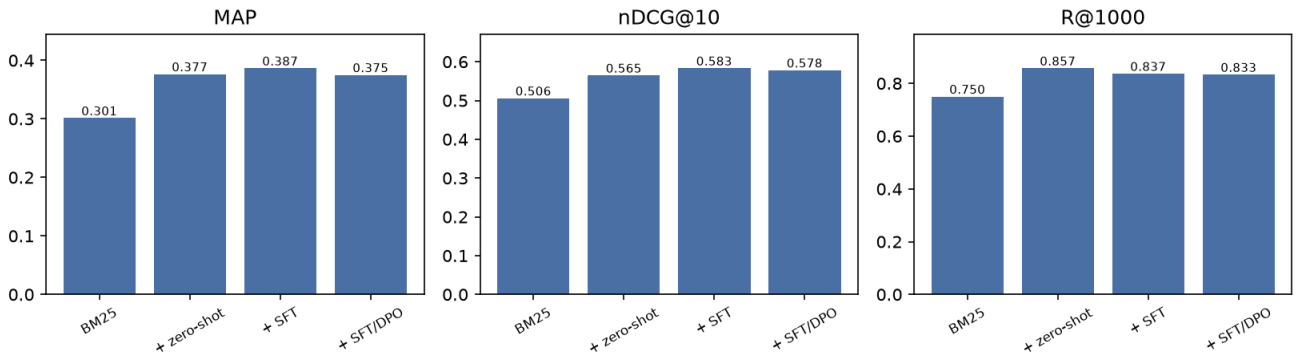


Figure 1: The three metrics across configurations. Essentially the entire improvement is attributable to expansion itself; the training stages are indistinguishable from one another.

4.2 Statistical significance

All figures are means over 43 queries, which is a small evaluation set by construction — deep pooled judgments are expensive. A difference in aggregate score is therefore not by itself evidence that one configuration is better than another. Table 4 reports paired t -tests over per-query scores, together with the win/loss/tie record.

Comparison	Metric	W/L/T	p	
BM25 $\rightarrow$ zero-shot	MAP	28/15/0	0.0006	significant
	nDCG@10	26/13/4	0.0653	not significant
	R@1000	22/7/14	0.0007	significant
zero-shot $\rightarrow$ SFT	MAP	22/20/1	0.5920	not significant
	nDCG@10	21/19/3	0.5275	not significant
	R@1000	10/8/25	0.2573	not significant
SFT $\rightarrow$ SFT/DPO	MAP	15/15/13	0.2806	not significant
	nDCG@10	14/15/14	0.7420	not significant
	R@1000	7/5/31	0.4494	not significant

Table 4: Paired t -tests between consecutive configurations, $n = 43$. Pairing is essential here: query difficulty varies far more between queries than between systems, and an unpaired test would be dominated by that variance.

Two results reach significance, and both come from the same comparison: expansion against no expansion. Every difference between training regimes falls between $p = 0.26$ and $p = 0.74$, with win/loss records close to even (22/20, 15/15, 14/15). These are not marginal results; they are indistinguishable from chance.

The nDCG@10 improvement from expansion deserves particular care. An 11.6% relative gain with 26 wins against 13 losses does not reach significance at $p = 0.065$, because the per-query differences are inconsistent in magnitude. We report it as a non-significant improvement rather than rounding the interpretation in our favour.

The detection threshold follows from the sample size. With per-query standard deviations of the observed magnitude, differences below approximately 0.05 absolute cannot be resolved at $n = 43$. The SFT advantage over zero-shot (0.010 MAP) and the DPO difference from SFT (0.012 MAP) both fall an order of magnitude below that threshold — and below the 0.017 run-to-run variance established in Section 3.

4.3 Generation statistics

Since the four configurations differ only in generated text, the properties of that text are the mechanism by which any difference arises. Table 5 reports mean length and vocabulary size against the corpus the models are trained to imitate.

	Mean words	× corpus mean	Distinct terms
MS MARCO passages	56.2	1.00	—
Zero-shot	96.8	1.72	1818
SFT	38.2	0.68	853
SFT/DPO	37.3	0.66	824

Table 5: Generated pseudo-document statistics against the corpus reference. Corpus figures are from a 1-in-500 sample of the collection ($n = 17,684$; median 50 words, p90 91 words). Distinct terms is the vocabulary pooled across all 43 evaluation queries. For scale, DL19 queries average 5.4 words.

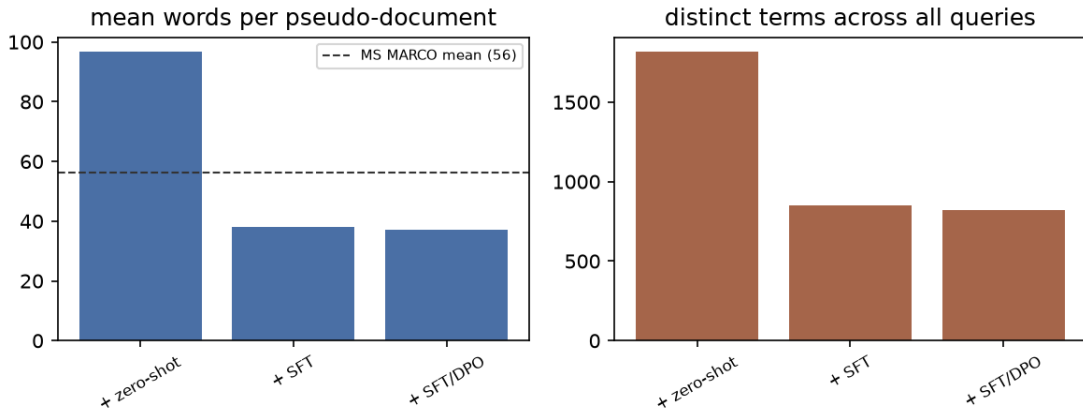


Figure 2: Mean generation length and pooled vocabulary. The dashed line marks the MS MARCO passage mean of 56.2 words. Zero-shot overshoots it; both fine-tuned configurations undershoot.

Supervised fine-tuning halves both length and vocabulary. Zero-shot generates 1.72 times the mean corpus passage length — above the corpus p90 of 91 words — while SFT generates 0.68 times. Both miss the corpus distribution, in opposite directions, and the configuration that overshoots has the higher Recall@1000.

DPO changes generation only marginally: 38.2 to 37.3 words and 853 to 824 distinct terms.

4.4 DPO preference data and training

Queries sampled	5,000
Preference pairs produced	3,628 (72.6% yield)
Discarded as tied	1,372 (27.4%)
Mean text similarity, chosen vs. rejected	0.073
Mean reciprocal-rank gap	0.225
Rejected candidate scoring zero	1,170 (32.2%)

Table 6: Preference construction statistics. Text similarity is the difflib sequence ratio over a 300-pair sample; a value near zero indicates the sampled candidates are substantially different text. The reciprocal-rank gap of 0.225 is approximately the difference between rank 1 and rank 4.

Two DPO runs were conducted. The first used the published defaults ($\text{lr} = 5 \times 10^{-6}$, $\beta = 0.1$) and produced no learning: training loss remained at $-\log 0.5 = 0.693$ throughout, reward margins alternated in sign, and preference

accuracy oscillated around chance.

Rather than adjusting hyperparameters immediately, we inspected the preference data to determine whether the failure lay in the signal or in the optimisation. Table 6 shows the pairs are well separated: chosen and rejected candidates share almost no text (similarity 0.073) and differ substantially in retrieval outcome (mean reciprocal-rank gap 0.225). This locates the failure in optimisation.

The second run raised the learning rate fourfold to 2×10^{-5} and halved β to 0.05. This produced learning: reward margins turned consistently positive and grew from 0.002 to 0.015, and training preference accuracy rose from 0.50 to 0.61. Held-out preference accuracy, however, reached only 0.52.

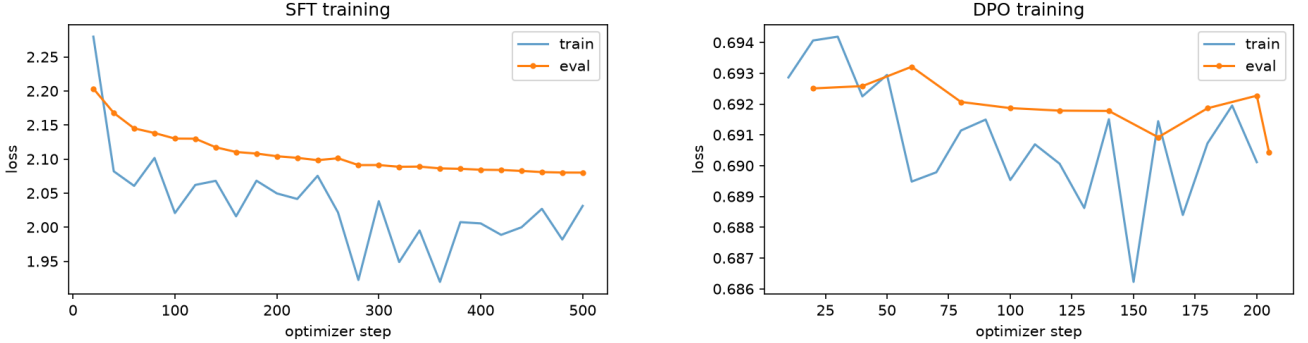


Figure 3: Left: SFT training and held-out loss. Training loss plateaus near step 100 while held-out loss continues to improve marginally to 2.018, which justifies the 8,000-example sample size. Right: DPO loss for the second run. Held-out loss remains close to 0.693 throughout, consistent with the held-out preference accuracy of 0.52.

The gap between training accuracy of 0.61 and held-out accuracy of 0.52 indicates that the model fitted the training preferences without learning a transferable one. Retrieval effectiveness fell slightly relative to SFT, though not significantly (MAP 0.3865 to 0.3750, $p = 0.28$).

5 Analysis

5.1 Expansion helps; the training regime does not

The clearest result is the shape of Table 4: significance appears in exactly one comparison, and it is the first one. Adding a generated pseudo-document to the query improves MAP by 25.0% ($p = 0.0006$) and Recall@1000 by 14.3% ($p = 0.0007$). Every subsequent comparison — zero-shot against SFT, SFT against DPO — is statistically indistinguishable from chance.

This is worth stating plainly because the aggregate numbers invite a different reading. SFT posts the best MAP and nDCG@10 in Table 3, and it would be easy to present that as an improvement. It is not one. The margin over zero-shot is 0.010 MAP, against a detection threshold of roughly 0.05 at $n = 43$ and a measured run-to-run variance of 0.017. The correct claim is that a 1.5B instruction-tuned model produces expansions of comparable utility whether or not it has been fine-tuned on the target corpus.

The Recall@1000 improvement is the more informative of the two significant results. Recall at depth 1000 measures whether relevant documents enter the candidate pool at all, and a 14.3% gain means expansion surfaces passages that the unexpanded query never retrieved at any depth. That is vocabulary mismatch being resolved, which is the mechanism Query2Doc claims. MAP rising more than nDCG@10 further indicates that the gains are distributed through the ranking rather than concentrated in the top ten.

5.2 Fine-tuning succeeded at the wrong objective

Both training stages worked, in the sense that both improved the quantity they were trained to improve.

SFT’s held-out loss decreased monotonically across all 500 steps, from 2.156 to 2.018, on queries the model never trained on. There is no overfitting here: the model generalised to unseen queries at every checkpoint. What it generalised was corpus style. Mean generation length moved from 96.8 words to 38.2, and the corpus reference in Table 5 shows where it was heading — from 1.72 times the MS MARCO passage mean toward it, overshooting to 0.68 times.

DPO likewise learned. In the second run, reward margins turned consistently positive and training preference accuracy rose from chance to 0.61.

Neither produced a measurable change in retrieval. The explanation is not that training failed but that the training objectives are proxies which diverge from the evaluated quantity:

Objective	Optimises
SFT	likelihood of one annotated passage
DPO	rank of one annotated passage
Evaluation	graded relevance across the top ten

The three correlate — all reward corpus-appropriate vocabulary — but they are not the same target, and hitting the first two did not move the third. We take this as the central finding: *distributional alignment with a corpus and utility as a BM25 expansion are separable properties of generated text*.

5.3 Why length matters for lexical retrieval

The generation statistics suggest a mechanism for the one directional difference that is not noise. BM25 matches terms, so a pseudo-document contributes value roughly in proportion to the useful terms it injects. Halving the length halves the vocabulary bridge.

Zero-shot holds the highest Recall@1000 at 0.8573 and writes 96.8 words with 1818 distinct terms pooled across the evaluation queries. SFT writes 38.2 words with 853 terms and drops to 0.8366. Fewer distinct terms means fewer entry points into the inverted index, so fewer novel documents enter the candidate pool.

Two observations sharpen this. First, the vocabulary-to-length ratio is slightly *higher* for SFT (853 terms over roughly 1640 total words, against 1818 over 4162), so SFT’s text is marginally less repetitive. Length, not verbosity, is the variable that changed. Second, zero-shot’s 96.8 words exceeds the corpus p90 of 91 words: its recall advantage comes from writing unnaturally long text rather than better text. This is not a property one would want to preserve deliberately, and it is a reason to be cautious about reading zero-shot’s recall as a virtue.

We note that the Recall@1000 difference is itself non-significant ($p = 0.26$), so this mechanism is offered as a consistent explanation of the observed direction rather than as a demonstrated effect. The intervention that would establish it is described in Section 7.

5.4 Why the retrieval-grounded reward did not transfer

Stage 3 exists precisely because SFT optimises the wrong objective, and its reward is grounded in retrieval rather than in likelihood. It also failed to transfer, which requires a separate explanation.

The reward is the reciprocal rank of *one* MS MARCO-judged passage. Consider two candidates for the same query, both reasonable expansions. One happens to surface the judged passage at rank 2; the other surfaces five relevant passages that were never judged, with the judged one at rank 40. The first wins. Whether it is the better expansion is a different question — the label reflects which passage an annotator happened to use, not which expansion retrieves better.

Under Table 1’s supervision regime, roughly one positive per query, a substantial fraction of the 3,628 preference labels therefore encode annotation artefacts rather than a generalisable property. A model can fit such labels on the training set — accuracy 0.61 — while finding nothing transferable underneath, which is what the held-out accuracy of 0.52 reports.

Two further factors compound this.

The graded signal is collapsed to an ordering. The mean reciprocal-rank gap between chosen and rejected is 0.225, and 32.2% of rejected candidates failed to retrieve the passage at all. That is a graded signal. DPO uses only the ordering: a pair with a gap of 0.9 contributes identically to one with a gap of 0.01. The near-tie pairs are the noisiest and receive equal weight.

DPO is structurally conservative. The β term penalises divergence from the reference policy, which here is the SFT checkpoint. DPO is designed to refine a policy rather than reverse it, so a large shift in generation behaviour was unlikely by construction. This is visible in the generation statistics: 38.2 words to 37.3, and 853 distinct terms to 824. The first run, at $\beta = 0.1$ and a learning rate four times lower, produced no shift at all.

5.5 What the hyperparameter sequence shows

The two DPO runs are reported together deliberately. The first, at published defaults, produced a null result that admits three explanations: the learning rate was too low, β was too high, or the preference pairs were not separable. The three imply different remedies, and guessing among them would have consumed several 40-minute runs.

Measuring the preference data (Table 6) eliminated the third explanation directly, which reduced the search to one adjustment. The second run confirmed that the constraint had been optimisation: identical data, relaxed hyperparameters, and learning where there had been none.

This does not establish that the second run’s hyperparameters are optimal. A grid over β and learning rate would be the standard approach and was not affordable here; we record this in Section 7 rather than implying a search that did not occur. What the sequence does establish is that the null result is a property of the method under this supervision, not an artefact of an untried learning rate.

6 Limitations

6.1 Evaluation

Forty-three queries. The binding limitation. Every reported figure is a mean over 43 judged queries, which places the resolution of this evaluation at roughly 0.05 absolute. Three of the differences in Table 3 fall an order of magnitude below that. Deep pooled judgments are expensive, so this is inherent to the test collection rather than a choice, but it means the experiment can distinguish expansion from no expansion and cannot distinguish training regimes from one another.

Single seed. Each configuration was trained and evaluated once. The two SFT runs described in Section 3 bound the variance at approximately 0.017 MAP, which exceeds the between-configuration differences. Three seeds per configuration would allow those differences to be tested properly; this was not affordable within the compute budget.

Pooling incompleteness. DL19 judgments were assembled by pooling the top results of 2019 systems [1]. Query expansion retrieves documents that those systems did not surface, and `trec_eval` scores an unjudged document as irrelevant. The expanded configurations may therefore be penalised for retrieving relevant passages that were never assessed. The direction of this bias is against the expansion methods, so it does not threaten the positive result, but it means the reported improvements are likely conservative.

6.2 Supervision

Shallow judgments. MS MARCO training labels are a by-product of an answer-writing task, not the output of relevance assessment. There is approximately one positive per query and no relevance grading. This caps how informative any training signal derived from them can be, and it is inherent to the dataset.

Single-positive simplification. Where a query has several judged positives we retain the first. Given the mean of approximately 1.0 positives per query this affects a small minority, and under a fixed sample budget query diversity is more valuable than depth per query for learning a stylistic prior. It is nonetheless a simplification that discards available supervision, and we record it as ours rather than as the dataset’s.

Unjudged is not irrelevant. The most damaging consequence of shallow supervision for Stage 3. A candidate that surfaces genuinely relevant but unlabelled passages is scored as a failure by Equation 2. The preference signal is thus not only sparse but occasionally inverted, and the model may be trained against good generations.

Ordinal collapse. As described in Section 5, a measured graded reward is reduced to a binary ordering before training. Preference-learning variants that weight pairs by reward magnitude would use the signal that was actually collected.

6.3 Method and implementation

No hyperparameter search. SFT hyperparameters are conventional LoRA values; DPO hyperparameters come from one diagnostic adjustment to published defaults. Neither was searched systematically. The reported DPO result is therefore evidence about one point in a space, not about the method’s best achievable behaviour.

Zero-shot generation is truncated. Zero-shot output averages 96.8 words, close to the ceiling implied by the 128-token cap fixed by the assignment. Its natural length is therefore longer than measured, and the comparison in Table 5 is between a capped configuration and two uncapped ones.

No chat template. An instruction-tuned model receives a bare completion prompt. This was chosen to keep the prompt identical across all three stages, as Section 2 describes, but it means the zero-shot configuration is not the model operating in its intended interface.

Reproducibility across library versions. The 0.017 MAP difference between two otherwise-identical SFT runs followed a change in library version and in the ordering of examples within batches. The two changes are confounded and we do not attribute the difference to either. It stands as an observation that results at this scale are sensitive to factors outside the documented configuration.

7 Conclusion

We implemented Query2Doc query expansion with Qwen2.5-1.5B-Instruct and evaluated three training configurations against an unexpanded BM25 baseline on TREC DL 2019 using a common retrieval and evaluation pipeline. Query expansion substantially improved retrieval: MAP increased by 25.0 ($p = 0.0006$) and Recall@1000 by 14.3improvement was not statistically significant at $n = 43$ ($p = 0.065$). The Recall@1000 improvement indicates that expansion retrieved relevant passages that the original queries failed to retrieve.

Neither SFT nor DPO produced a measurable retrieval improvement over the zero-shot configuration. SFT successfully aligned generation with the MS MARCO passage distribution, while DPO achieved a training preference accuracy of 0.61, but these improvements in their respective training objectives did not translate into better retrieval. This suggests that corpus-distribution alignment and preference optimization are not necessarily aligned with the lexical retrieval objective of Query2Doc.

Overall, the results show that a lightweight 1.5B open-weight model can provide effective query expansion on a laptop, with the clearest benefit appearing in Recall@1000. The lack of improvement from fine-tuning further suggests that better generation or preference objectives do not automatically lead to better retrieval effectiveness.

References

- [1] Nick Craswell, Bhaskar Mitra, Emine Yilmaz, Daniel Campos, and Ellen M. Voorhees. Overview of the trec 2019 deep learning track. *arXiv preprint arXiv:2003.07820*, 2020. URL <https://arxiv.org/abs/2003.07820>.
- [2] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *International Conference on Learning Representations*, 2022. URL <https://arxiv.org/abs/2106.09685>.
- [3] Jimmy Lin, Xueguang Ma, Sheng-Chieh Lin, Jheng-Hong Yang, Ronak Pradeep, and Rodrigo Nogueira. Pyserini: A python toolkit for reproducible information retrieval research with sparse and dense representations. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 2356–2362, 2021. doi: 10.1145/3404835.3463238.
- [4] Tri Nguyen, Mir Rosenberg, Xia Song, Jianfeng Gao, Saurabh Tiwary, Rangan Majumder, and Li Deng. Ms marco: A human generated machine reading comprehension dataset. *arXiv preprint arXiv:1611.09268*, 2016. URL <https://arxiv.org/abs/1611.09268>.
- [5] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D. Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems*, volume 36, pages 53728–53741, 2023. URL https://papers.nips.cc/paper_files/paper/2023/hash/a85b405ed65c6477a4fe8302b5e06ce7-Abstract-Conference.html.
- [6] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: Bm25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009. doi: 10.1561/15000000019.
- [7] Qwen Team. Qwen2.5: A party of foundation models, September 2024. URL <https://qwenlm.github.io/blog/qwen2.5/>.
- [8] Liang Wang, Nan Yang, and Furu Wei. Query2doc: Query expansion with large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 9414–9423, Singapore, 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.585. URL <https://aclanthology.org/2023.emnlp-main.585/>.